Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering &Technology

ESTD:1995

Department of Computer Science & Engineering

SUBJECT: Computer Networks

UNIT II

Topic Name: **Data Link Layer and Medium Access Sub Layer**:
III B.Tech CSE A

Dr.Muniraju Naidu Vadlamudi
Assistant Professor
Email: munirajunaidu_v@vnrvjiet.in

Agenda

ESTD:1995

- Error Detection and Error Correction
  - Fundamentals
  - Block coding
  - Hamming Distance
  - CRC
- Flow Control and Error control protocols
  - Stop and Wait,
  - Go back – N ARQ,
  - Selective Repeat ARQ,
  - Sliding Window,
  - Piggybacking,
- Random Access &Multiple access protocols –
  - Pure ALOHA, Slotted ALOHA, CSMA/CD,CDMA/CA

Data Link Layer

ESTD:1995

- The data link layer transforms the physical layer, a raw transmission facility, to a link responsible for node-to-node (hop-to-hop) communication.
- Responsibilities of the data link layer include *framing, addressing, flow control, error control,* and *media access control.*
- The data link layer divides the stream of bits received from the network layer into manageable data units called frames.
- The data link layer adds a header to the frame to define the addresses of the sender and receiver of the frame.
- Imposes a flow control mechanism to avoid overwhelming the receiver.
- Adds reliability to the physical layer by adding mechanisms to detect and retransmit damaged, duplicate, or lost frames.
- When two or more devices are connected to the same link, data link layer protocols are necessary to determine which device has control over the link at any given time.

Error Detection and Correction

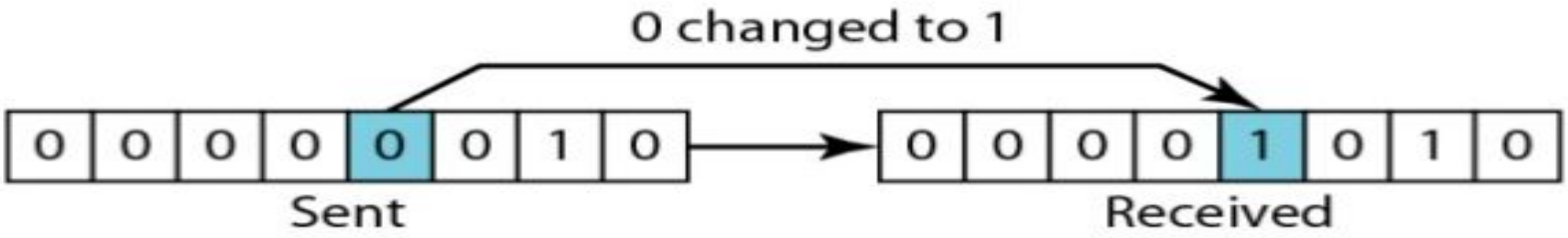
ESTD:1995

- Networks must be able to transfer data from one device to another with acceptable accuracy.
- For most applications, a system must guarantee that the data received are identical to the data transmitted.
- Message can be altered due to many factors.
- Some applications can tolerate a small level of error. For example, random errors in audio or video transmissions may be tolerable, but when transferring text , high accuracy is expected.
- Types of Errors
  - Whenever bits flow from one point to another, they are subject to unpredictable changes because of interference.
    - Single bit error
    - Burst error

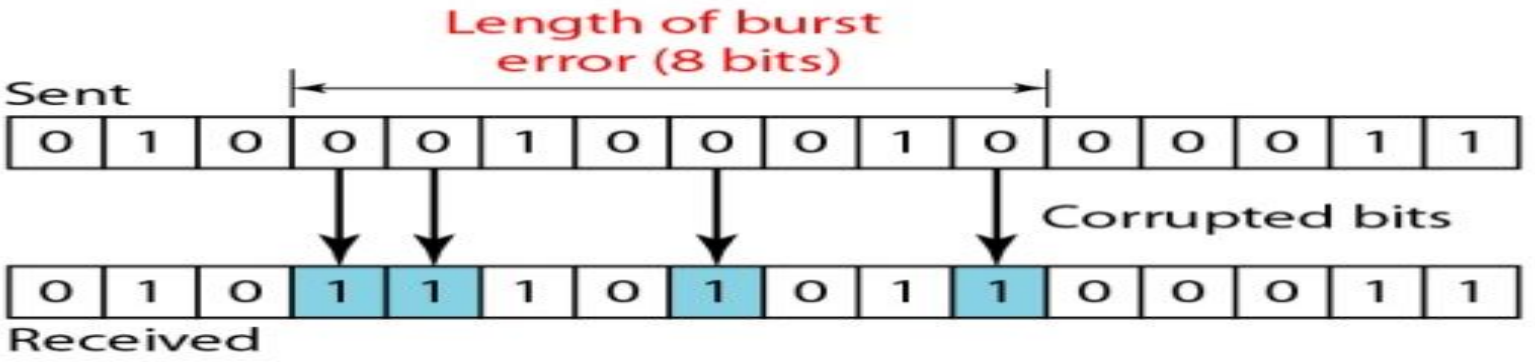
Error Detection and Correction

ESTD:1995

Single bit error



Burst error of length 8



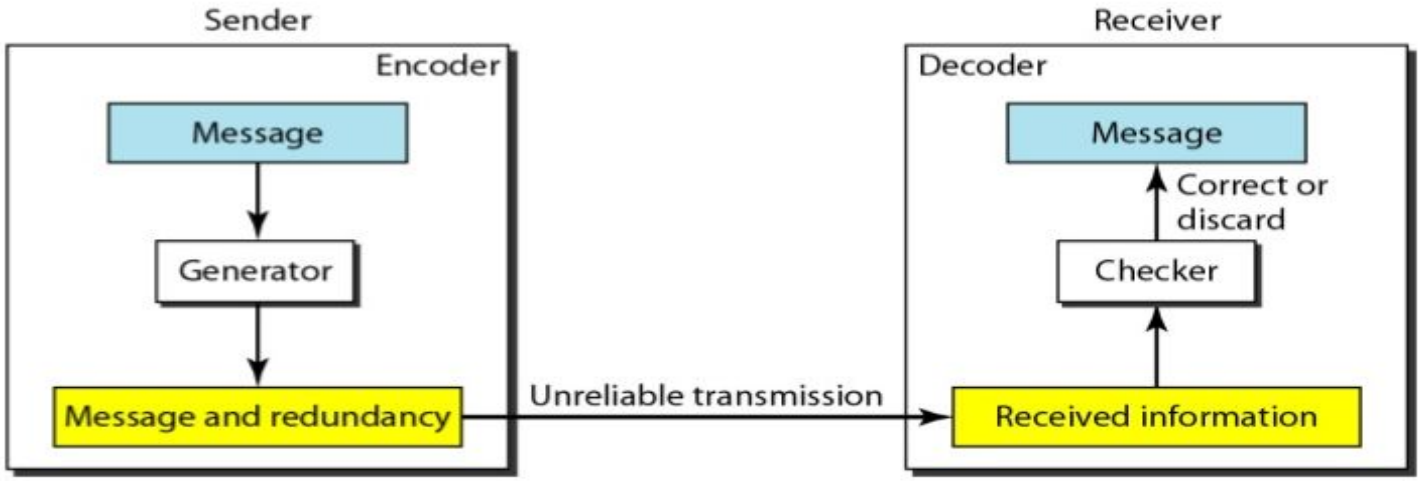
Error Detection and Error Correction

- Redundancy
  - To able to detect or correct errors, there is a need to send some extra bits with our data. These redundant bits are added by the sender and removed by the receiver. Their presence allows the receiver to detect or correct corrupted bits.

- Forward Error Correction Versus Retransmission
  - There are two main methods of error correction. Forward error correction is the process in which the receiver tries to guess the message by using redundant bits.
  - Correction by retransmission is a technique in which the receiver detects the occurrence of an error and asks the sender to resend the message. Resending is repeated until a message arrives that the receiver believes is error-free.

Error Detection and Error Correction

- **Coding**
  - The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits. The receiver checks the relationships between the two sets of bits to detect or correct the errors.

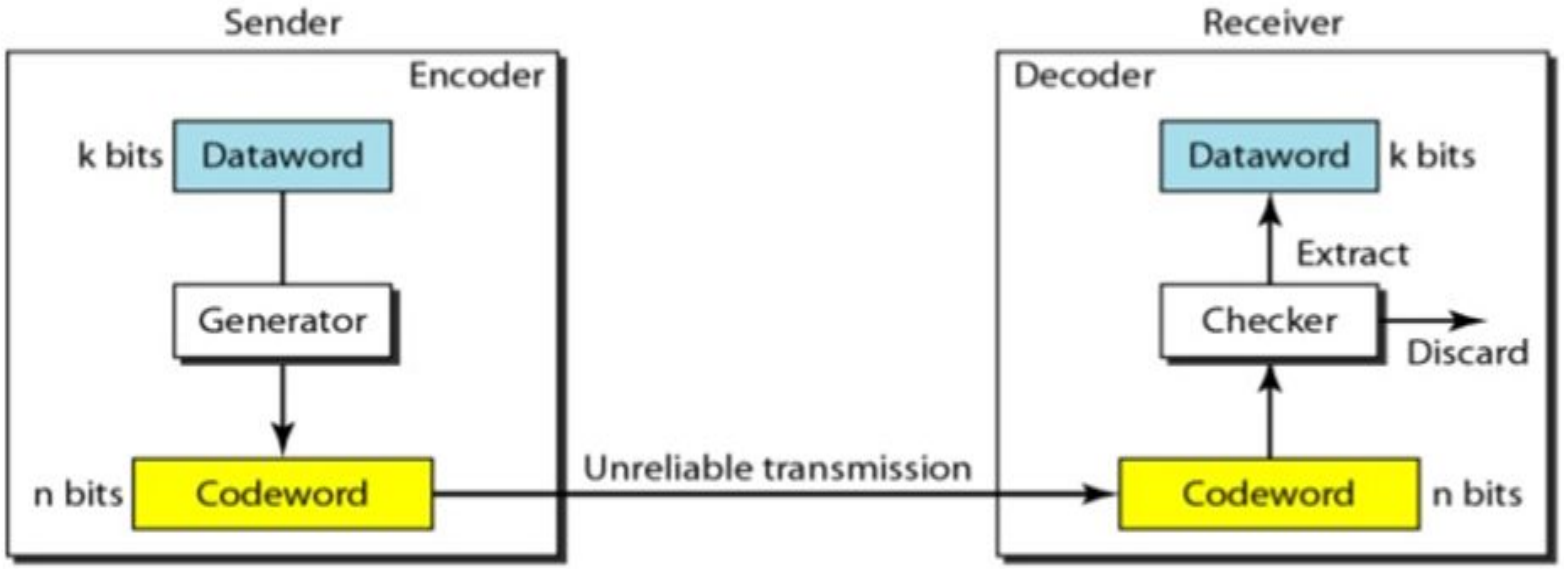


Error Detection and Error Correction

- **Modular Arithmetic**
  - In modular arithmetic, we use only a limited range of integers. An upper limit is defined called a modulus *N.*
  - use only the integers 0 to *N* - I, inclusive. This is *modulo-N* arithmetic.
  - In this arithmetic XOR (exclusive OR) operation is used for both addition and subtraction. The result of an XOR operation is 0 if two bits are the same; the result is I if two bits are different.

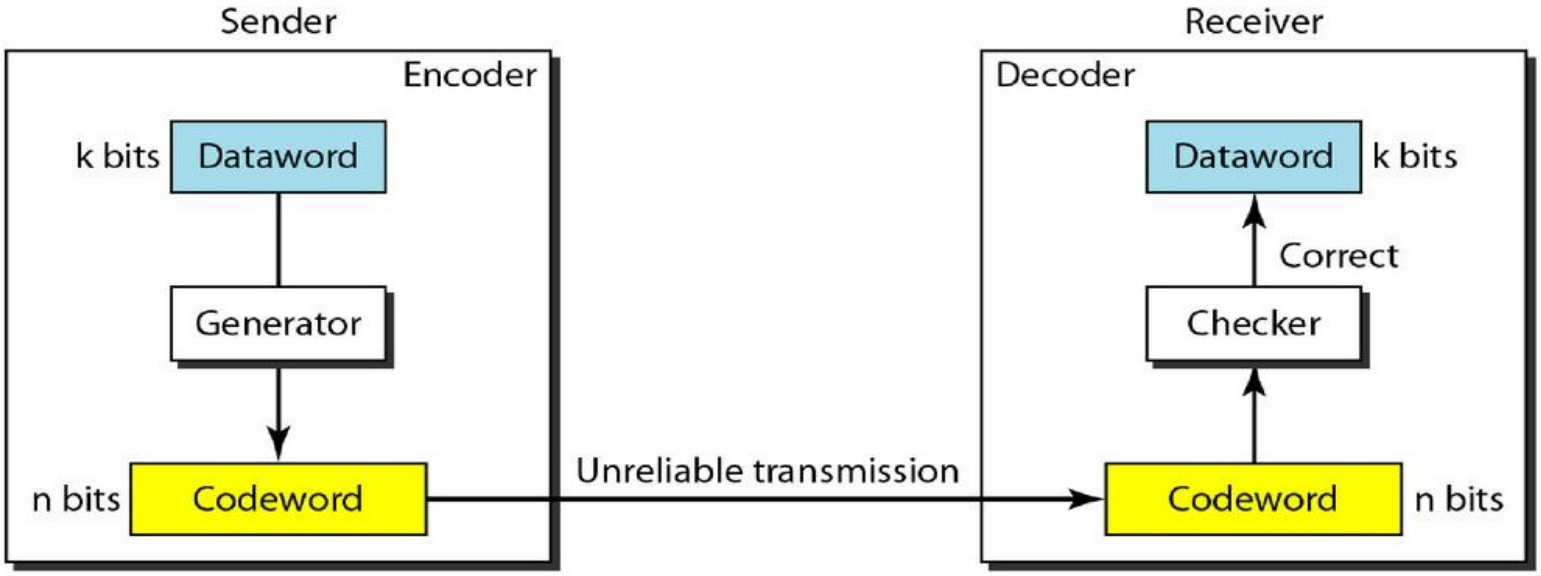
Error Detection and Error Correction

- **BLOCK CODING**
  - In block coding, message is divided into blocks, each of *k* bits, called data words. add *r* redundant bits to each block to make the length *n* = *k* + *r.* The resulting *n-bit* blocks are called code words.
- ***Process of error detection in block coding***



Error Detection and Error Correction

- **Error Correction**
  - Error correction is difficult than error detection. In error detection, the receiver needs to know only that the received code word is invalid; in error correction the receiver needs to find the original code word sent. There is a need for more redundant bits for error correction than for error detection.



Error Detection and Error Correction

ESTD:1995

- **Hamming Distance**
  - The Hamming distance between two words (of the same size) is the number of differences between the corresponding bits and represented by **d(x,y).**
  - The Hamming distance can easily be found by applying the XOR operation ($\oplus$) on the two words
  - Note that the Hamming distance is a value greater than zero.
    - For example the Hamming distance **d(10101, 11110)** is 3 because 10101 $\oplus$ 11110 is 01011 (three 1's).
- The minimum Hamming distance is the smallest Hamming distance between all possible pairs. Let d_{min} is used to define the minimum Hamming distance in a coding scheme.
  - *For example:* d(000,011)=2 d(000,111)=3 . The d_{min} in this case is 2.

Error Detection and Error Correction



- ***Hamming Distance and Error***
  - The Hamming distance between the received code-word and the sent code-word is the number of bits that are corrupted during transmission.
- ***Minimum Distance for Error Detection***
  - If **s** errors occur during transmission, the Hamming distance between the sent code-word and received code-word is **s**. To detect up to **s errors**, the minimum distance between the valid codes must be **s+1**,

Error Detection and Error Correction

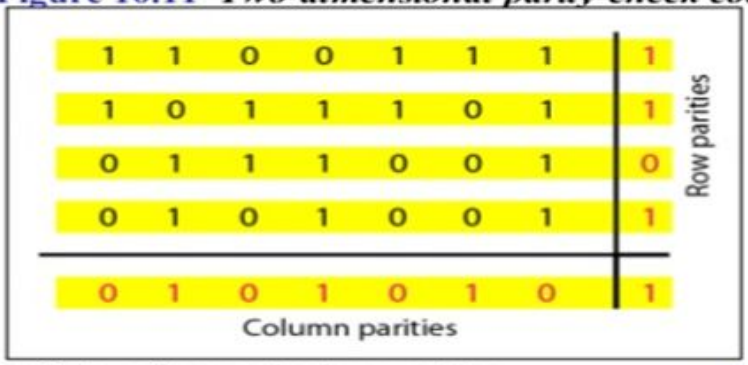
ESTD:1995

- **LINEAR BLOCK CODES**
  - All block codes used today belong to a subset called linear block codes.
  - use of nonlinear block codes for error detection and correction is not as widespread because their structure makes theoretical analysis and implementation difficult.
  - A linear block code is a code in which the exclusive OR (addition modulo-2) of two valid code-words creates another valid code-word.
  - Simple to find the minimum Hamming distance for a linear block code.
- ***Simple Parity-Check Code***
  - A *k-bit* data word is changed to an n-bit code word where $n = k + 1$.
  - Parity bit(odd parity or even parity)
  - The minimum Hamming distance for this category is $\mathbf{d_{min} = 2}$, which means that the code is a single-bit error-detecting code; it cannot correct any error.

Error Detection and Error Correction

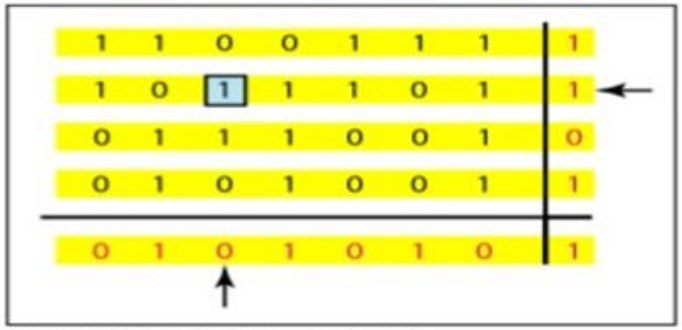
- In two-dimensional parity check, the data-word is organized in a table (rows and columns)

Figure 10.11 *Two-dimensional parity-check code*

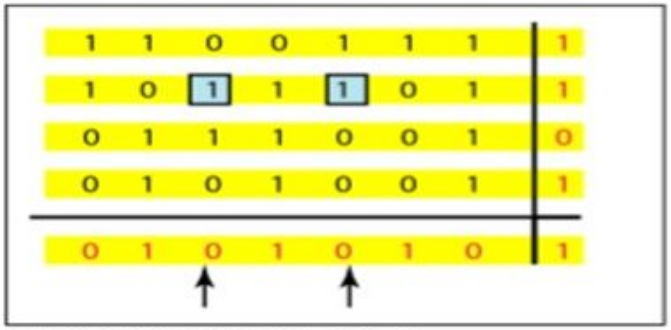


a. Design of row and column parities

Figure 10.11 *Two-dimensional parity-check code*

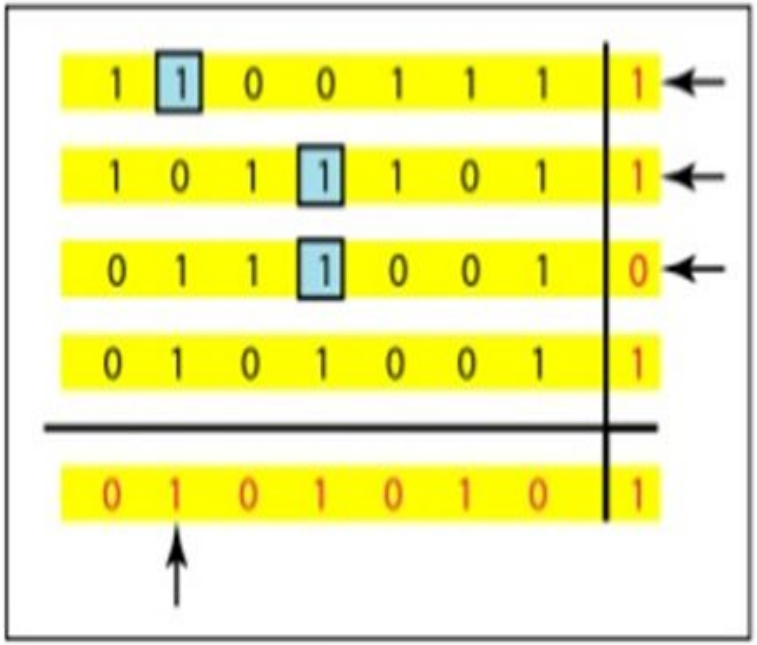


b. One error affects two parities

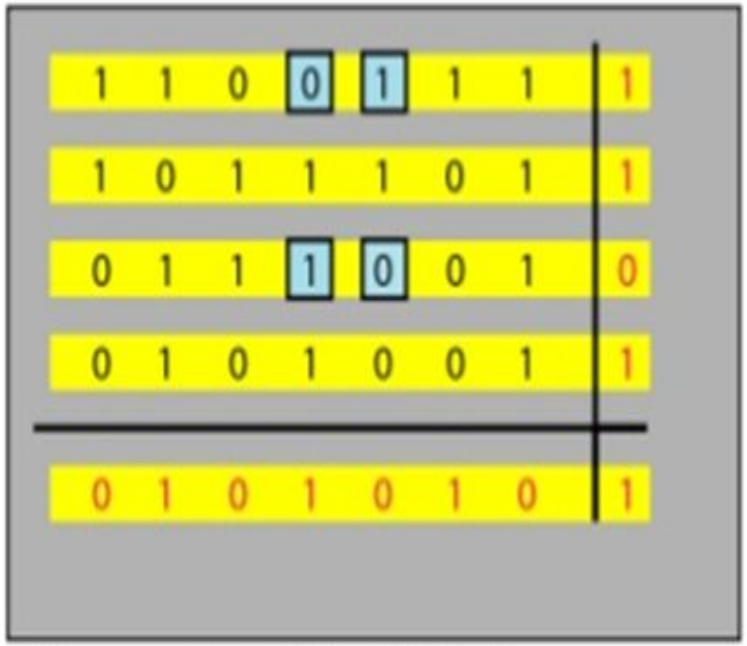


c. Two errors affect two parities

Error Detection and Error Correction



d. Three errors affect four parities



e. Four errors cannot be detected

Error Detection and Error Correction

ESTD:1995

Datawords	Codewords	Datawords	Codewords
0000	0000000	1000	1000110
0001	0001101	1001	1001011
0010	0010111	1010	1010001
0011	0011010	1011	1011100
0100	0100011	1100	1100101
0101	0101110	1101	1101000
0110	0110100	1110	1110010
0111	0111001	1111	1111111

Generator

r0=a2+a1+a0 (modulo-2)

r1 =a3 + a2 + a1 (modulo-2)

r2=a1 +a0+a3 (modulo-2)

Checker

S0=b2+b1+b0+ q0 (modulo-2)

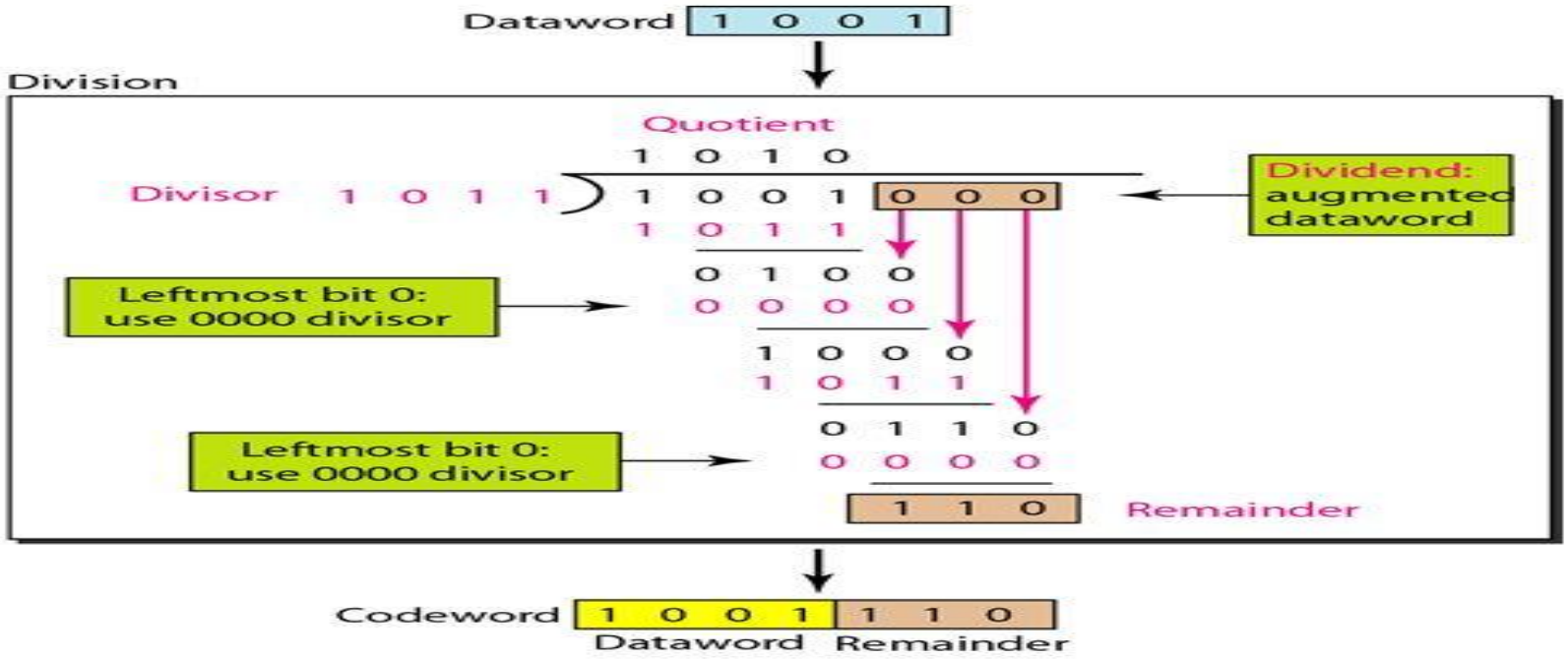
S1 =b3 +b2 +b1+ q1 (modulo-2)

S2=b1 +b0++b3+q2 (modulo-2)

Error Detection and Error Correction

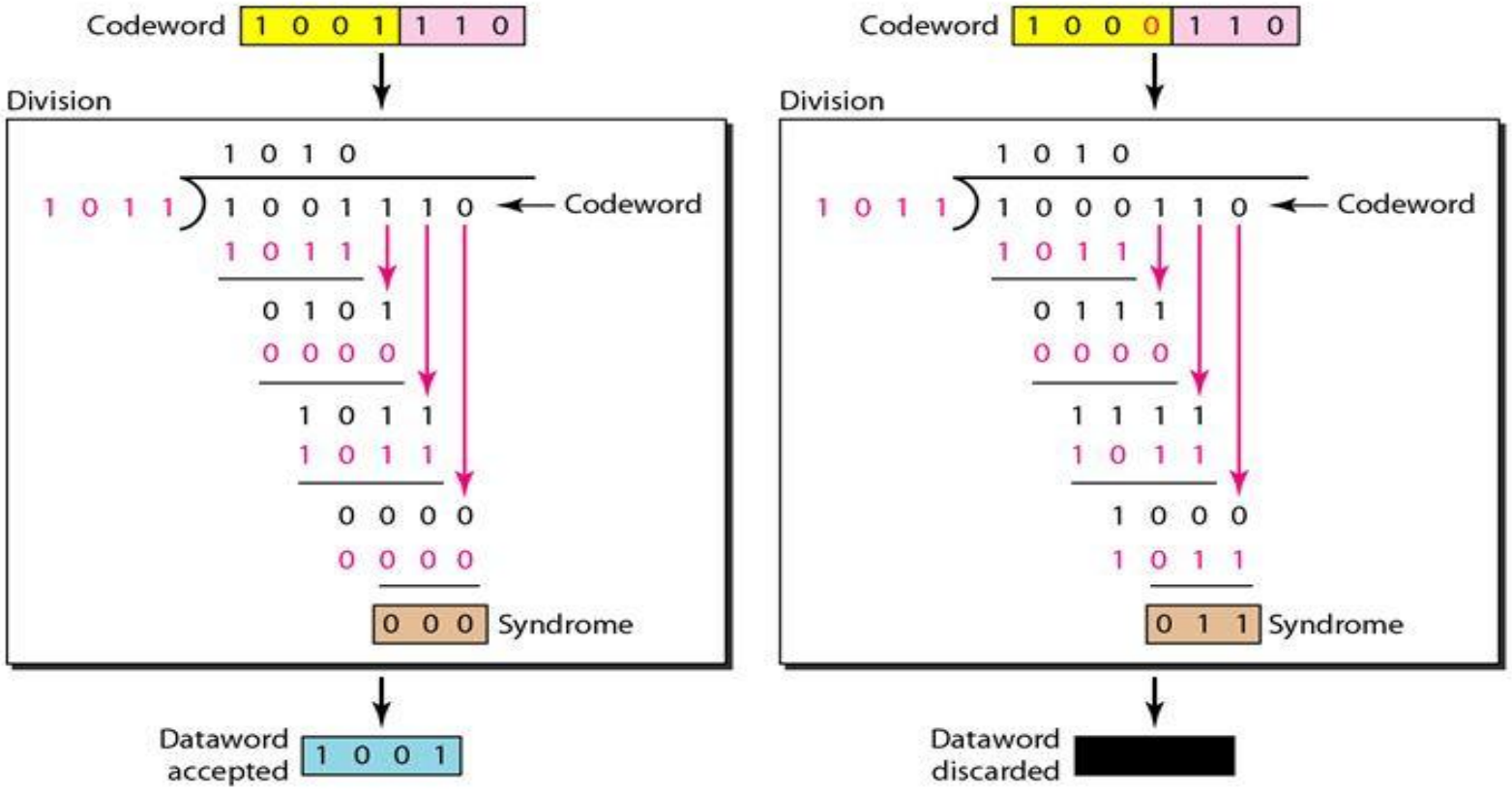
- **CYCLIC CODES**
  - Cyclic codes are special linear block codes with one extra property. In a cyclic code, if a code-word is cyclically shifted (rotated), the result is another code-word.

Encoder



Error Detection and Error Correction

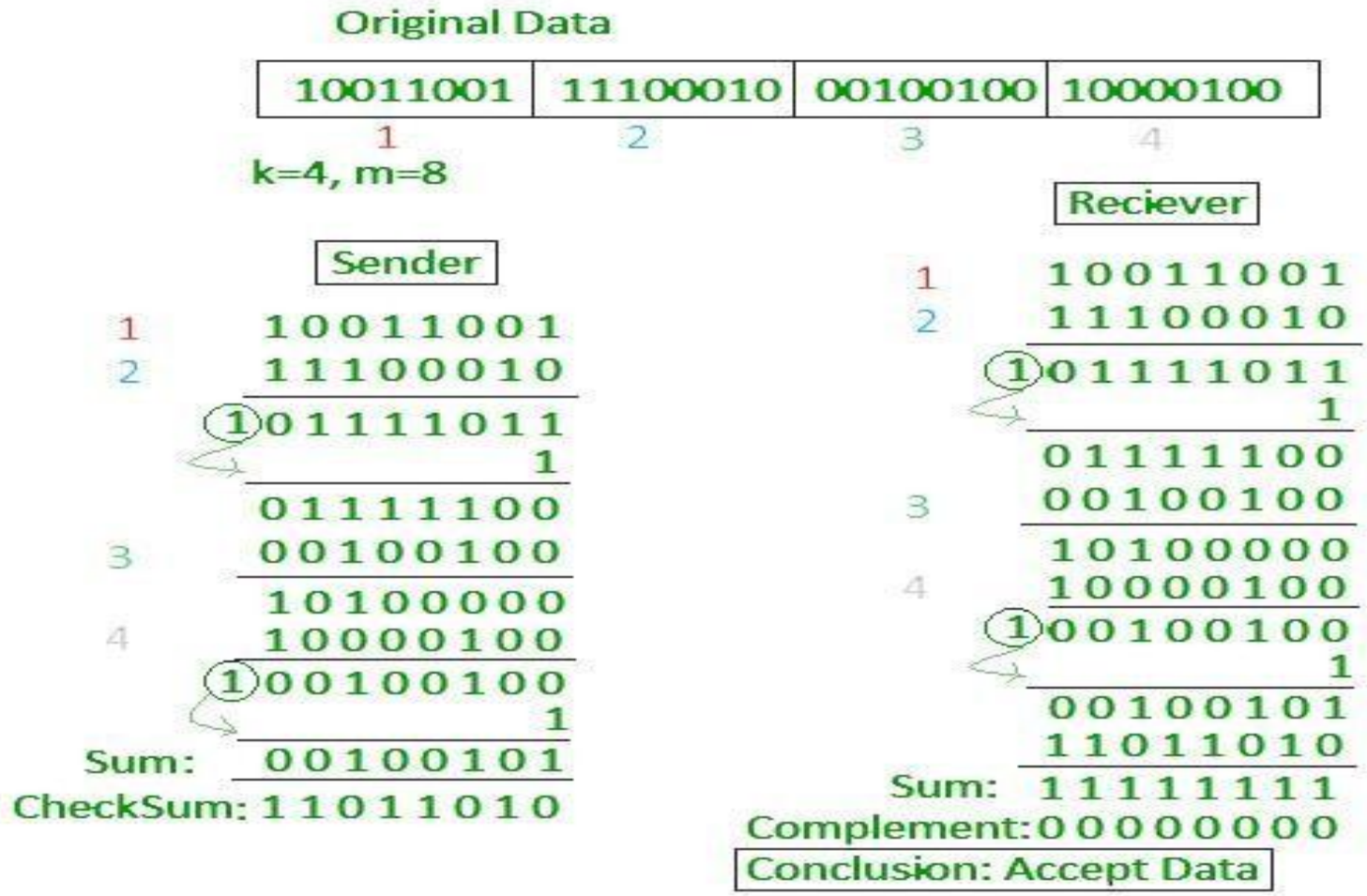
- Decoder



Error Detection and Error Correction

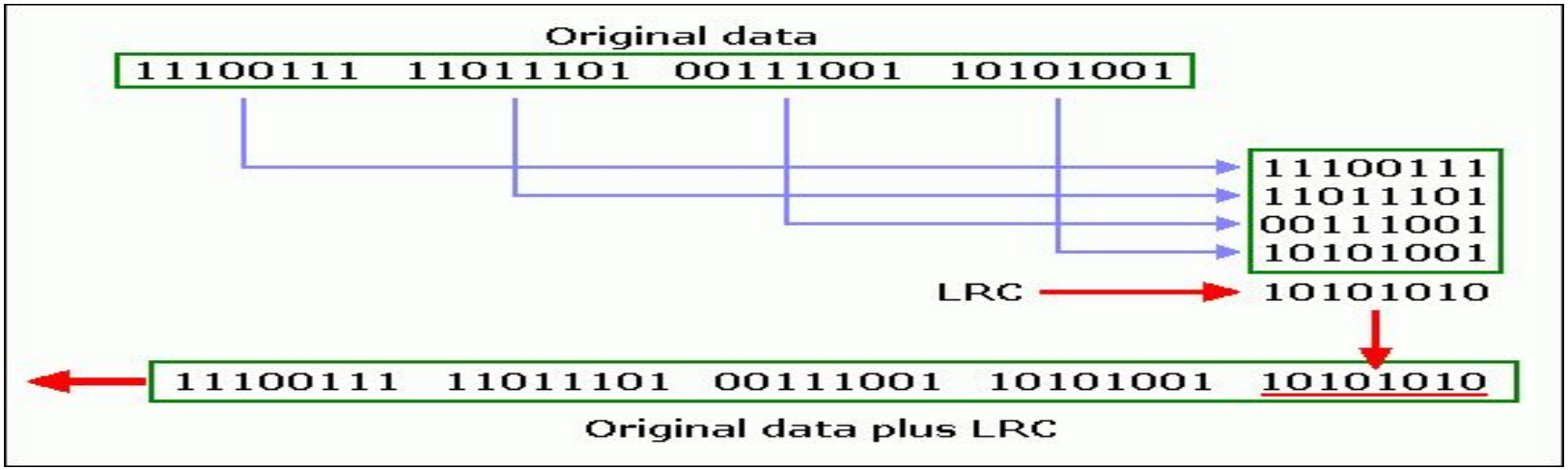
- **Checksum**
  - In checksum error detection scheme, the data is divided into k segments each of m bits.
  - In the sender's end the segments are added using 1's complement arithmetic to get the sum. The sum is complemented to get the checksum.
  - The checksum segment is sent along with the data segments.
  - At the receiver's end, all received segments are added using 1's complement arithmetic to get the sum. The sum is complemented.
  - If the result is zero, the received data is accepted; otherwise discarded.

Error Detection and Error Correction



Error Detection and Error Correction

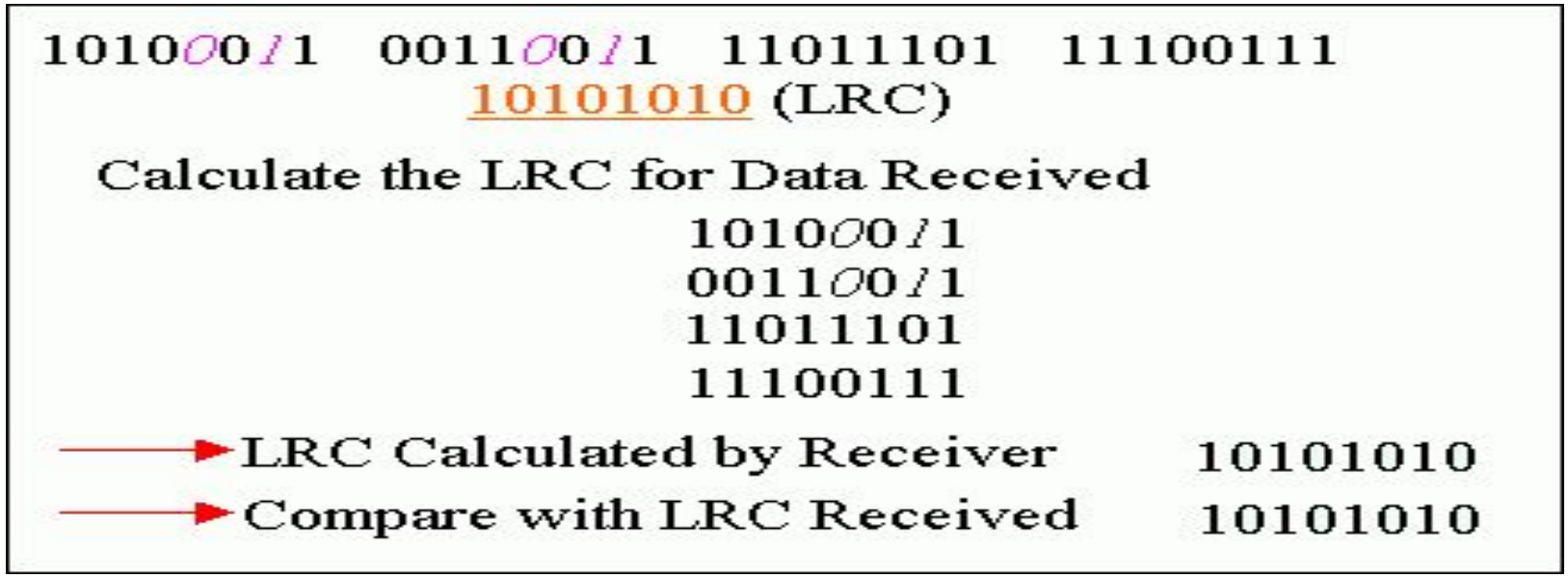
- **Longitudinal Redundancy Check (LRC)**
  - In this error detection method, a block of bits is organized in a table with rows and columns.
  - Then the parity bit for each column is calculated and a new row of eight bits, which are the parity bits for the whole block, is created.
  - After that the new calculated parity bits are attached to the original data and sends to the receiver.



Error Detection and Error Correction

ESTD:1995

- An LRC of *n* bits can easily detects a burst error of *n* bits.
- If two bits in one data unit are damaged and two bits in exactly the same positions in another data unit are also damaged, the LRC checker will not detect an error.



How LRC Fail to Detect the Burst Noise

Error Detection and Error Correction

- *Standard Polynomials*

Name	*Polynomial*	*Application*
CRC-8	$x^8 + x^2 + x + 1$	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	LANs

Data Link Control

ESTD:1995

- The two main functions of the data link layer are node-to-node communication and media access control, i.e., how to share the link.
- **Data link control** functions include framing, flow and error control, and software implemented protocols that provide smooth and reliable transmission of frames between nodes.
- There are five protocols: two for noiseless (ideal) channels and three for noisy (real) channels.
- **FRAMING**
  - Data transmission in the physical layer means moving bits in the form of a signal from the source to the destination.
  - The physical layer provides bit synchronization to ensure that the sender and receiver use the same bit durations and timing
- **Fixed-Size Framing:** In fixed-size framing, there is no need for defining the boundaries of the frames; the size itself can be used as a delimiter.

Data Link Control

ESTD:1995

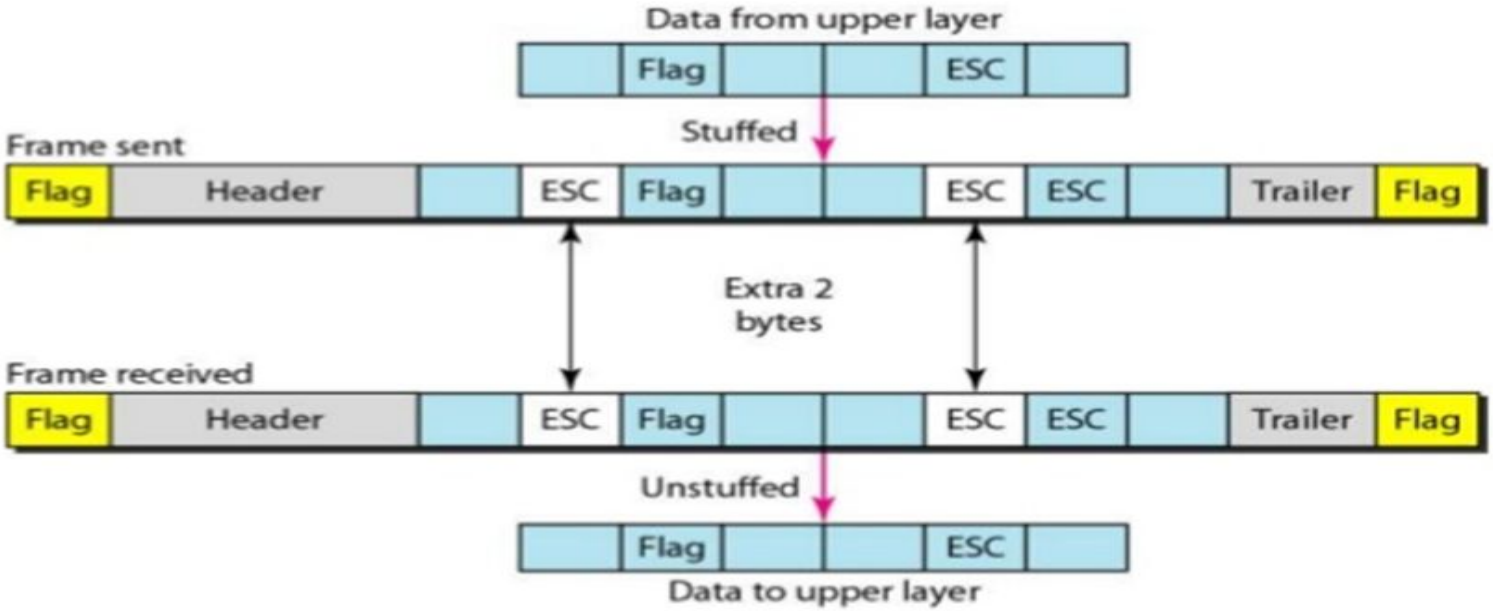
- **Variable-Size Framing**
  - Variable-size framing is prevalent in local area networks.
  - In variable-size framing, there is a need to define the end of the frame and the beginning of the next frame.
  - There are two approaches were used for this purpose: a character-oriented approach and a bit-oriented approach.
- ***Character-Oriented Protocols***
  - In a character-oriented protocol, data to be carried are 8-bit characters from a coding system such as ASCII .
  - The header, which normally carries the source and destination addresses and other control information, and the trailer, which carries error detection or error correction redundant bits, are also multiples of 8 bits.
  - To separate one frame from the next, an 8-bit (I-byte) flag is added at the beginning and the end of a frame. The flag, composed of protocol-dependent special characters, signals the start or end of a frame.

Data Link Control

ESTD:1995

- Character-oriented framing was popular when only text was exchanged by the data link layers.
- The flag could be selected to be any character not used for text communication. Sending other types of information such as graphs, audio, and video may result in the **flag** could also be part of the information.
- If this happens, the receiver, when it encounters this pattern in the middle of the data, thinks it has reached the end of the frame.
- To fix this problem, a byte-stuffing strategy was added to character-oriented framing. a special byte is added to the data section of the frame when there is a character with the same pattern as the flag. The data section is stuffed with an extra byte.
- This byte is usually called the escape character **(ESC),** which has a predefined bit pattern.
- What happens if the text contains one or more escape characters followed by a flag?

Data Link Control-Character stuffing

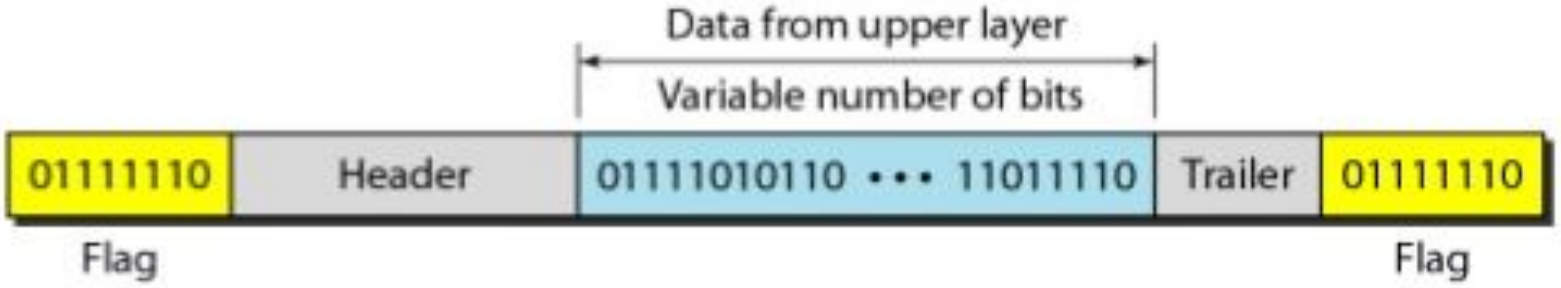


Character-oriented protocols present another problem The universal coding systems in use today, such as Unicode, have 16-bit and 32-bit characters that conflict with 8-bit characters. Hence, there is a tendency to move towards the bit-oriented protocols

Data Link Control-Bit stuffing

ESTD:1995

- ***Bit-Oriented Protocols***
  - In a bit-oriented protocol, in addition to headers (and possible trailers), there is a need for delimiter to separate one frame from the other.
  - Most protocols use a special 8-bit pattern flag 01111110 as the delimiter to define the beginning and the end of the frame.
  - If the flag pattern appears in the data, there is a need to somehow inform the receiver that this is not the end of the frame.



Data Link Control-Bit stuffing

ESTD:1995

- This can be done by stuffing 1 single bit (instead of 1 byte) to prevent the pattern from looking like a flag called bit stuffing.
- In bit stuffing, if 0 and five consecutive 1 bits are encountered, an extra 0 is added.
- This extra stuffed bit is eventually removed from the data by the receiver. Note that the extra bit is added after one 0 followed by five 1s regardless of the value of the next bit.
- This guarantees that the flag field sequence does not inadvertently appear in the frame.

Data Link Control- Flow Control

ESTD:1995

- Data communication requires at least two devices working together, one to send and the other to receive.
- The most important responsibilities of the data link layer are flow control and error control.
- Flow control is a set of procedures that tells the sender how much data it can transmit before it must wait for an acknowledgment from the receiver.
- Any receiving device has a limited speed for process incoming data and a limited amount of memory to store incoming data.
- The receiving device must be able to inform the sending device before those limits are reached .
- Incoming data must be checked and processed before they can be used.
- Each receiving device has a block of memory, called a *buffer,* reserved for storing incoming data until they are processed.

Data Link Control- Error Control

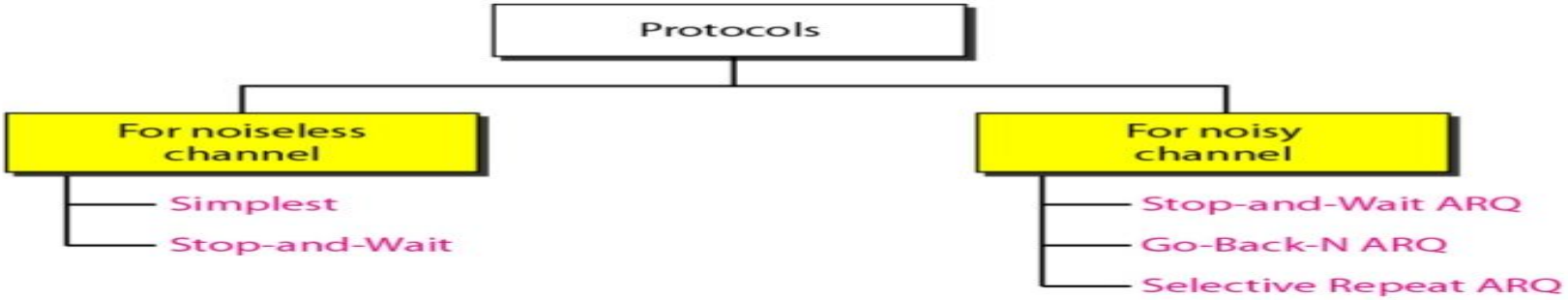
ESTD:1995

- **Error Control**
  - Error control is both error detection and error correction.
  - It allows the receiver to inform the sender of any frames lost or damaged in transmission and coordinates the retransmission of those frames by the sender.
  - Any time an error is detected in an exchange, specified frames are retransmitted and this process is called automatic repeat request (ARQ).

Data Link Control- Taxonomy of protocols

ESTD:1995

- The datalink layer combines framing, flow control, and error control to achieve the delivery of data from one node to another.
- The protocols are normally implemented in software by using one of the common programming languages.
- Protocols are divided into those that can be used for noiseless (error-free) channels and those that can be used for noisy (error-creating) channels.



Data Link Control- Simplex protocol

ESTD:1995

- The first protocol is called Simplest Protocol that has no flow or error control.
- It is a unidirectional protocol in which data frames are traveling in only one direction-from the sender to receiver.
- Receiver can immediately handle any frame it receives with a processing time that is small enough to be negligible.
- The data link layer of the receiver immediately removes the header from the frame and hands the data packet to its network layer, which can also accept the packet immediately.

```
1  while(true)                    // Repeat forever
2  {
3   WaitForEvent();               // Sleep until an event occurs
4   if(Event(RequestToSend))      //There is a packet to send
5   {
6     GetData();
7     MakeFrame();
8     SendFrame();                //Send the frame
9   }
10 }
```

Data Link Control- Simplex protocol

```
1  while(true)                          // Repeat forever
2  {
3    WaitForEvent();                    // Sleep until an event occurs
4    if(Event(ArrivalNotification))     //Data frame arrived
5    {
6       ReceiveFrame();
7       ExtractData();
8       DeliverData();                  //Deliver data to network layer
9    }
10 }
```

Data Link Control- Stop-and-Wait Protocol

ESTD:1995

- If data frames arrive at the receiver site faster than they can be processed, the frames must be stored until their use.
- If the receiver does not have enough storage space it may result in either the discarding of frames or denial of service.
- Need to tell the sender to slow down.
- The protocol called Stop-and-Wait Protocol is used to maintain flow control.
- The sender sends one frame, stops until it receives confirmation from the receiver (ACK).
- Unidirectional communication for data frames, but auxiliary ACK frames travel from the other direction.

Data Link Control- Stop-and-Wait Protocol

ESTD:1995

```
1  while(true)                          //Repeat forever
2  canSend = true                       //Allow the first frame to go
3  {
4    WaitForEvent();                    // Sleep until an event occurs
5    if(Event(RequestToSend) AND canSend)
6    {
7       GetData();
8       MakeFrame();
9       SendFrame();                    //Send the data frame
10      canSend = false;                //Cannot send until ACK arrives
11   }
12   WaitForEvent();                    // Sleep until an event occurs
13   if(Event(ArrivalNotification)      // An ACK has arrived
14    {
15      ReceiveFrame();                 //Receive the ACK frame
16      canSend = true;
17    }
18 }
```

At Sender Site

Data Link Control- Stop-and-Wait Protocol

ESTD:1995

```
1  while(true)                          //Repeat forever
2  {
3    WaitForEvent();                    // Sleep until an event occurs
4    if(Event(ArrivalNotification))     //Data frame arrives
5    {
6       ReceiveFrame();
7       ExtractData();
8       Deliver(data);                  //Deliver data to network layer
9       SendFrame();                    //Send an ACK frame
10   }
11 }
```

At Receiver site

Data Link Control- Noisy Channels

ESTD:1995

Stop-and-Wait Automatic Repeat Request

- When the frame arrives at the receiver site, it is checked and if it is corrupted, it is silently discarded. The detection of errors in this protocol is manifested by the silence of the receiver.
- Lost frames are more difficult to handle than corrupted ones.
- In previous protocols, there was no way to identify a frame. The received frame could be the correct one, or a duplicate, or a frame out of order.
- The solution is to number the frames. When the receiver receives a data frame that is out of order, this means that frames were either lost or duplicated.

Data Link Control- Noisy Channels

ESTD:1995

- The receiver does not respond when there is an error, how can the sender know which frame to resend? To remedy this problem, the sender keeps a copy of the sent frame.
- At the same time, it starts a timer. If the timer expires and there is no ACK for the sent frame, the frame is resent, the copy is held, and the timer is restarted.
- Since the protocol uses the stop-and-wait mechanism, there is only one specific frame that needs an ACK even though several copies of the same frame can be in the network.
- Since an ACK frame can also be corrupted and lost, it too needs redundancy bits and a sequence number and hence a sequence number field is required.

Data Link Control- Noisy Channels

- ***Sequence Numbers:*** *If* the field is m bits long, the sequence numbers start from 0, go to $2^m - 1$ and then are repeated.
- Let us use x a sequence number; we only need to use *x* + 1 after that. No need for *x* + 2. To show this, assume that the sender has sent the frame numbered *x*. Three things can happen.
  1. The frame arrives correctly to the receiver site; the receiver sends an acknowledgment. The acknowledgment arrives at the sender site, causing the sender to send the next frame numbered *x* + 1.
  2. The frame arrives correctly to the receiver site; the receiver sends an acknowledgment, but the acknowledgment is corrupted or lost. The sender resends the frame (numbered *x)* after the time-out. Note that the frame here is a duplicate. The receiver can recognize this fact because it expects frame *x* + I but frame *x* was received.
  3. The frame is corrupted or never arrives at the receiver site; the sender resends the frame (numbered *x)* after the time-out.

Data Link Control- Noisy Channels

ESTD:1995

- In **Stop-and-Wait ARQ** uses sequence numbers to number the frames.
- The sequence numbers are based on modulo-2 arithmetic.
- In Stop-and-Wait ARQ, the acknowledgment number always announces the sequence number of the next frame to be expected(Modulo-2)
- However this protocol wastes the bandwidth.

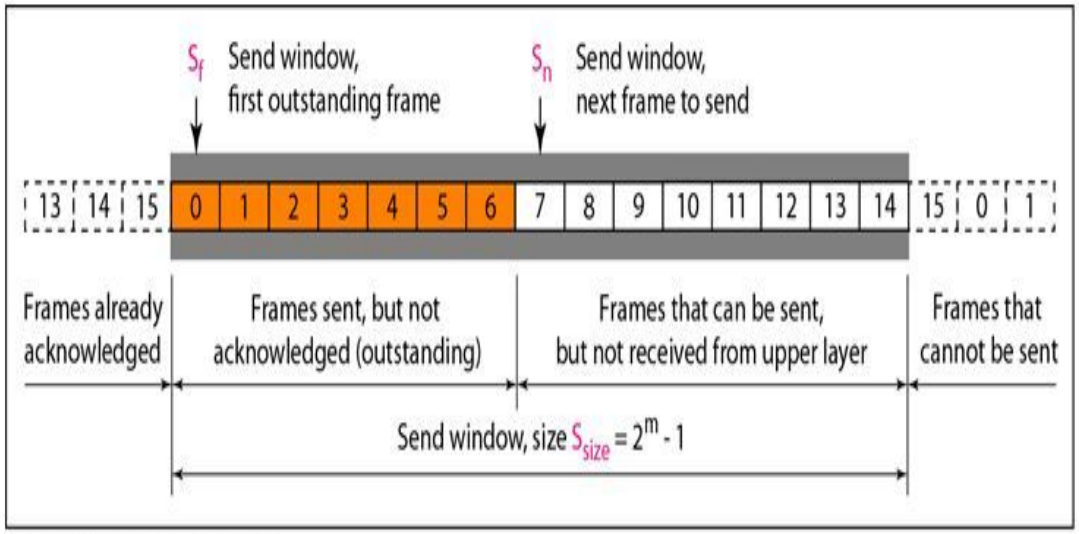
- ***Pipelining***
  - In networking and in other areas, a task is often begun before the previous task has ended. This is known as pipelining.
  - no pipelining in Stop-and-Wait ARQ because , the current frame is to be acknowledged before the next frame can be sent.
  - However, pipelining does apply to our next two protocols because several frames can be sent before receiving the acknowledgements about the previous frames .
  - Pipelining improves the efficiency of the transmission if the number of bits in transition is large with respect to the bandwidth-delay product.

Go-Back-N Automatic Repeat Request

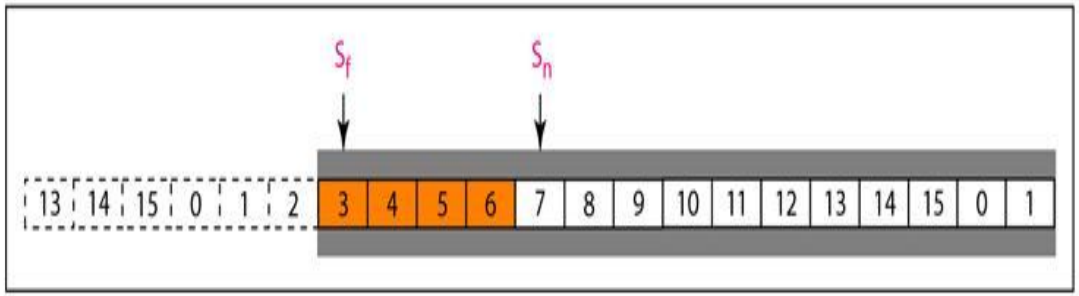
ESTD:1995

- To improve the efficiency of transmission (filling the pipe), multiple frames must be in transition while waiting for acknowledgment.
- In Go-Back-N Automatic Repeat Request several frames can be sent before receiving acknowledgments;
- A copy of these frames were kept until the acknowledgments arrive.
- ***Sliding Window***
  - ❑ The sliding window is an abstract concept that defines the range of sequence numbers that is the concern of the sender and receiver.
  - ❑ In each window position, some of these sequence numbers define the frames that have been sent; others define those that can be sent. The maximum size of the window is $2^m - 1$.

Go-Back-N Automatic Repeat Request



a. Send window before sliding

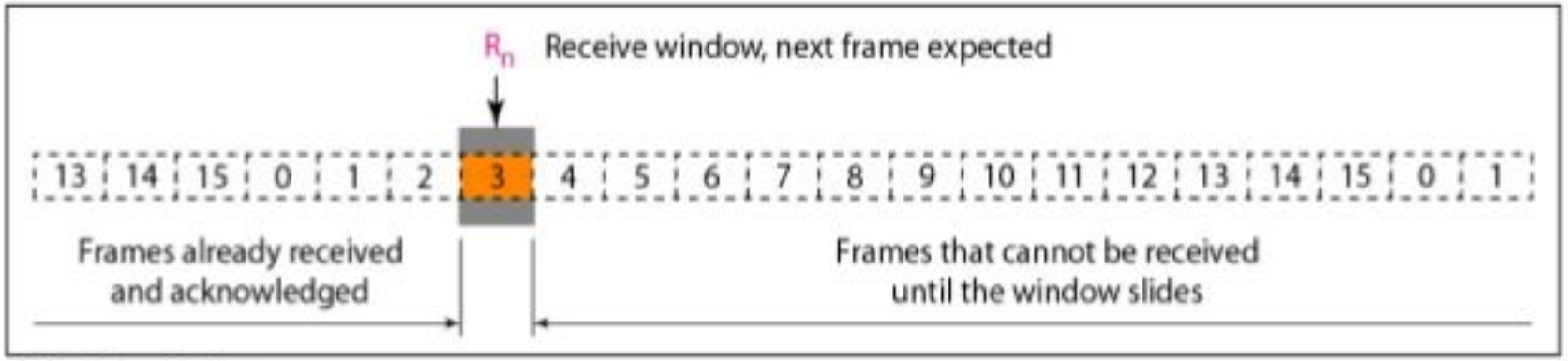


b. Send window after sliding

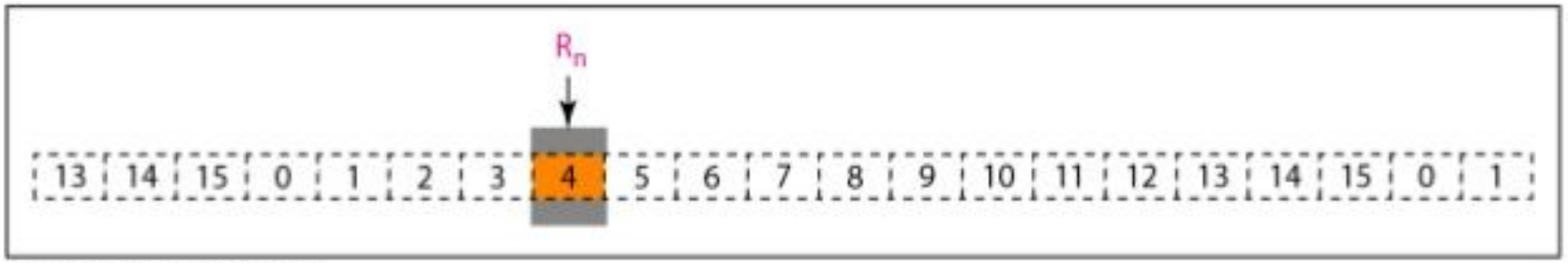
It is possible to acknowledge more than one frame. In Figure 11.12b, frames 0, I, and 2 are acknowledged, so the window has slid to the right three slots. Note that the value of S_f is 3 because frame 3 is now the first outstanding frame.

Go-Back-N Automatic Repeat Request

- There is a need of only one variable $\mathbf{R_n}$ (receive window, next frame expected) to define this abstraction.



a. Receive window

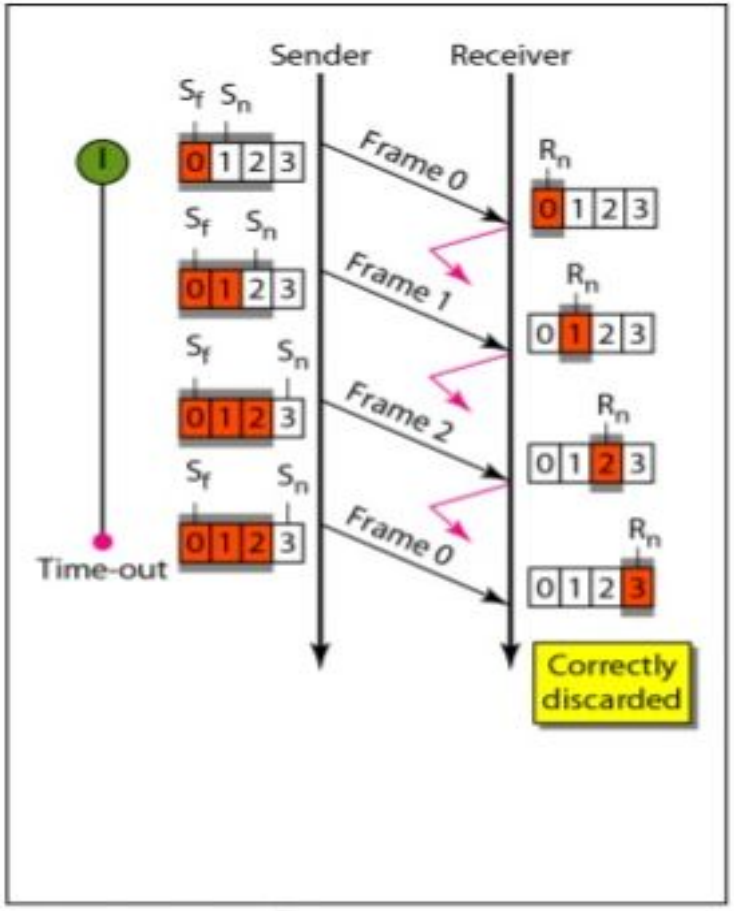


b. Window after sliding

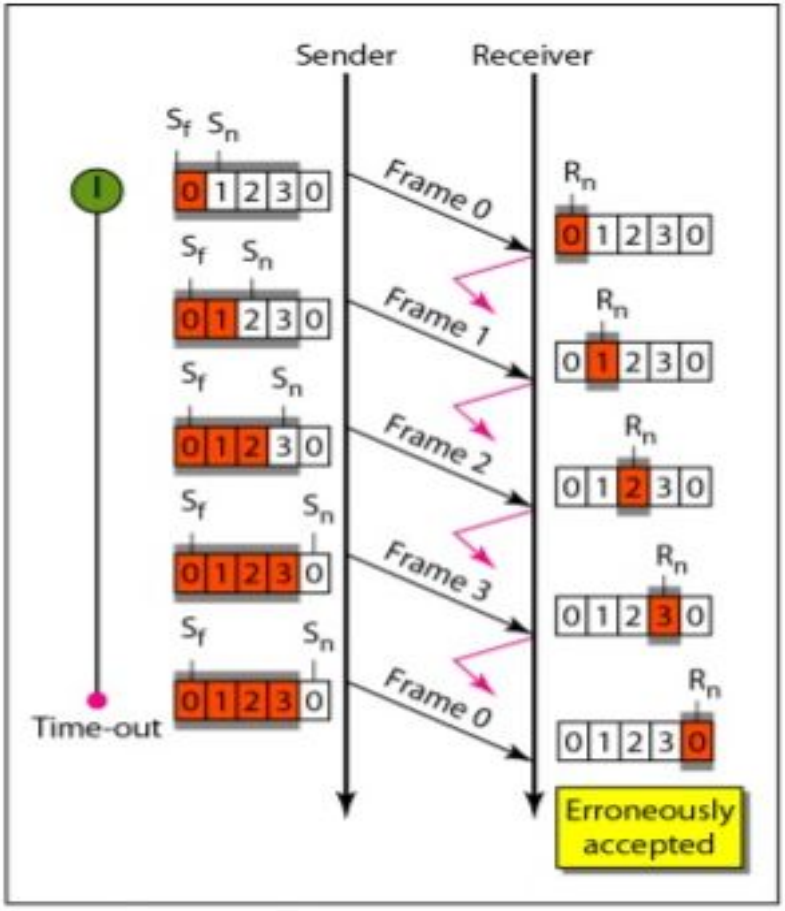
Go-Back-N Automatic Repeat Request

- *Timers:* There can be a timer for each frame that is sent, we send all outstanding frames when this timer expires.
- *Acknowledgment :* The receiver sends a positive acknowledgment if a frame has arrived safe and in order. If a frame is damaged or is received out of order, the receiver is silent and will discard all subsequent frames until it receives the one it is expecting. The silence of the receiver causes the timer of the unacknowledged frame at the sender site to expire.
- *Resending a Frame:* When the timer expires, the sender resends all outstanding frames. For example, suppose the sender has already sent frame 6, but the timer for frame 3 expires. This means that frame 3 has not been acknowledged; the sender goes back and sends frames 3, 4,5, and 6 again. That is why the protocol is called *Go-Back-N* ARQ.

Go-Back-N Automatic Repeat Request



a. Window size < 2^m



b. Window size = 2^m

Utilization of GO Back N

ESTD:1995

Pipelined protocols are an improvement over the stop-and-wait protocol. To achieve better network utilization, we have multiple frames "in-flight" between the sender and the receiver at a given time.

N denotes the window size in Go-Back-N, which the sender is allowed to send before receiving an acknowledgment. Basically if N=1 , we have a stop-and-wait implementation.

The maximum possible link utilization formula, disregarding any overheads is: utilization <= $\frac{N}{1+2*BD}$

In the formula, **BD** is the bandwidth-delay product, representing how much data can be carried over the link at a given time. It's calculated as the data link's capacity multiplied by round-trip delay time.

Selective Repeat Automatic Repeat Request

ESTD:1995

- *Go-Back-N* ARQ simplifies the process at the receiver site.
- The receiver keeps track of only one variable, and there is no need to buffer out-of-order frames; they are simply discarded.
- However, this protocol is very inefficient for a noisy link.
- In a noisy link a frame has a higher probability of damage, which means the resending of multiple frames.
- This resending uses up the bandwidth and slows down the transmission.
- For noisy links, there is another mechanism that does not resend *N* frames when just one frame is damaged called **selective repeat**

Selective Repeat Automatic Repeat Request

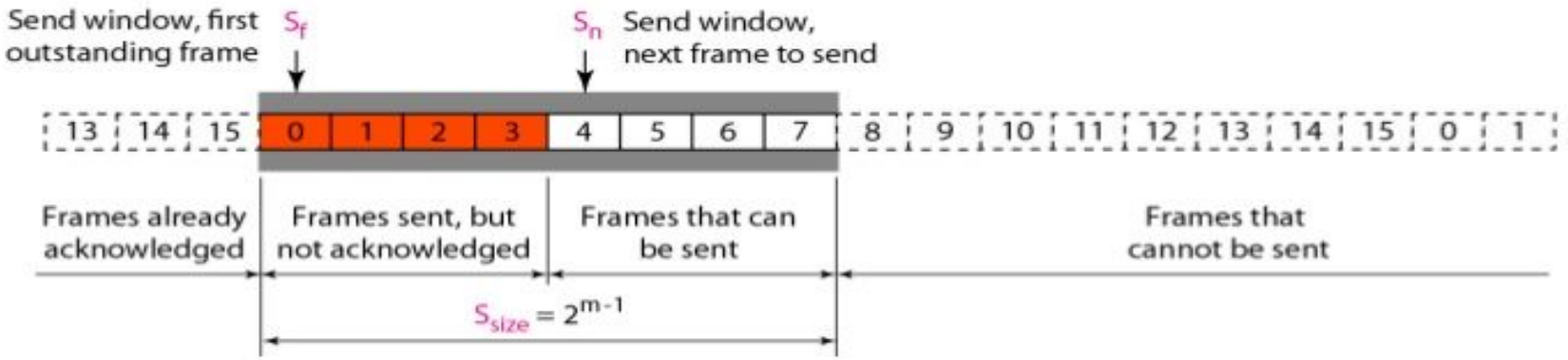
- *Go-Back-N* ARQ simplifies the process at the receiver site.
- The receiver keeps track of only one variable, and there is no need to buffer out-of-order frames; they are simply discarded.
- However, this protocol is very inefficient for a noisy link.
- In a noisy link a frame has a higher probability of damage, which means the resending of multiple frames.
- This resending uses up the bandwidth and slows down the transmission.
- For noisy links, there is another mechanism that does not resend *N* frames when just one frame is damaged; only the damaged frame is resent.
- This mechanism is called **Selective RepeatARQ**. It is more efficient for noisy links, but the processing at the receiver is more complex.

Selective Repeat Automatic Repeat Request

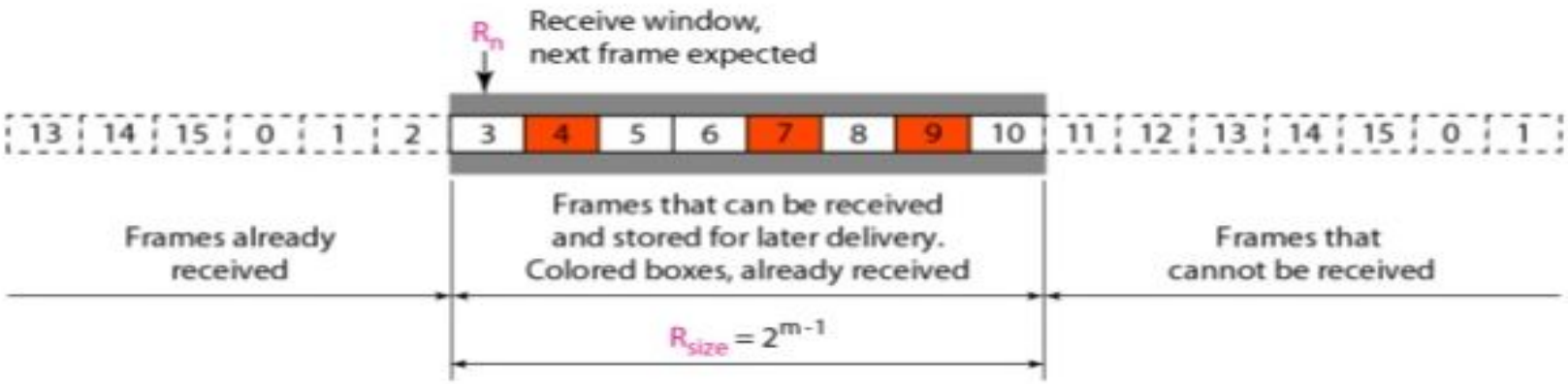
ESTD:1995

- *Windows:* The Selective Repeat Protocol also uses two windows: **a send window and a receive window.** The **send window maximum size can be 2^m-1**. The size of **the receive window is the same as the size of the send window which is 2^m-1.**

Send window for Selective Repeat ARQ



Selective Repeat Automatic Repeat Request



Selective Repeat Automatic Repeat Request

ESTD:1995

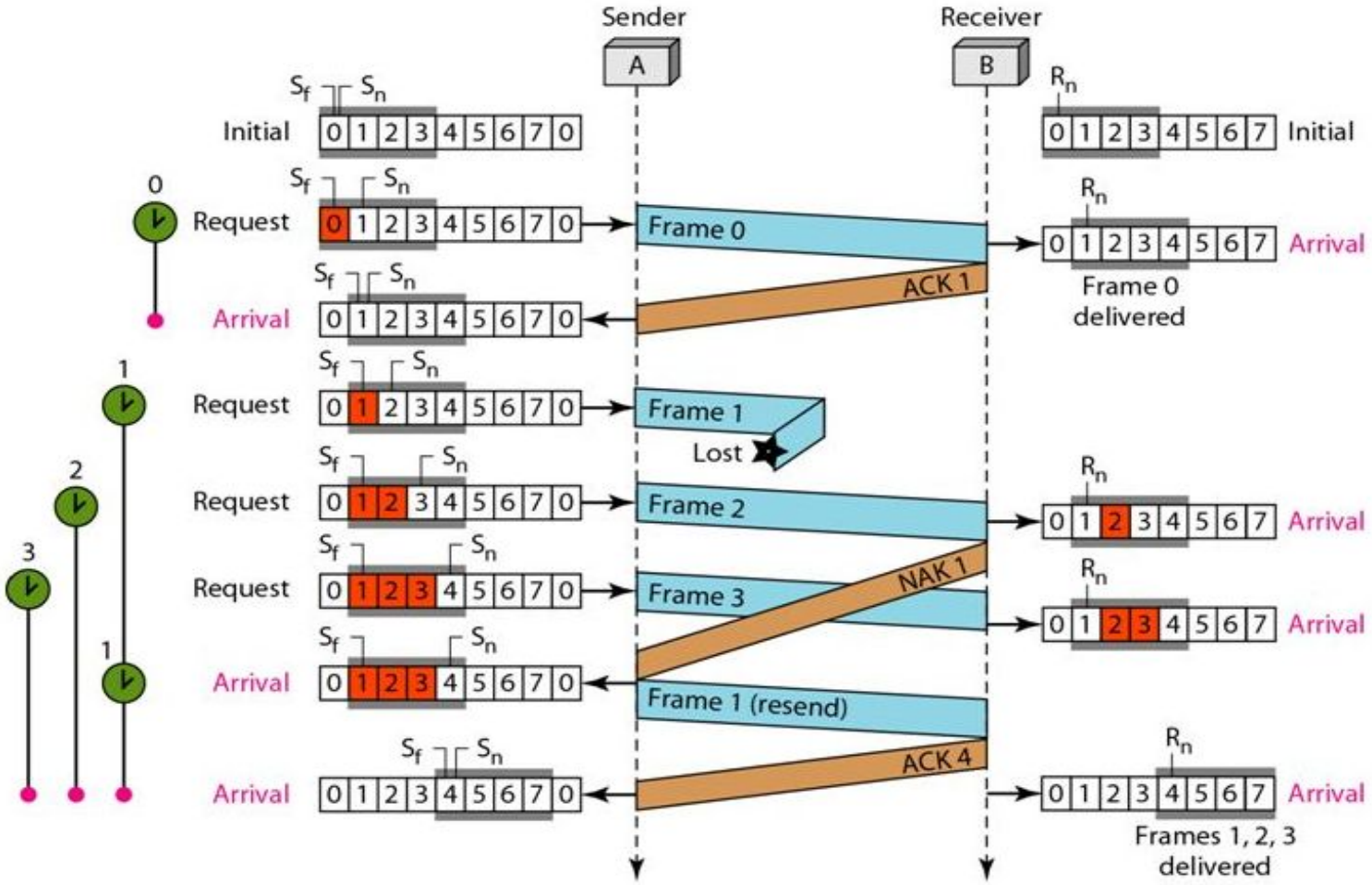
- ***Window Sizes***
  - The sender and receiver windows must be at most one half of 2^m
  - If the size of the window is 2 and all acknowledgments are lost, the timer for frame 0 expires and frame 0 is resent.
  - However, the window of the receiver is now expecting frame 2, not frame 0, so this duplicate frame is correctly discarded.
  - When the size of the window is 3 and all acknowledgments are lost, the sender sends a duplicate of frame 0.
  - However, this time, the window of the receiver expects to receive frame 0 (0 is part of the window), so it accepts frame 0, not as a duplicate, but as the first frame in the next cycle. This is clearly an error.

Selective Repeat Automatic Repeat Request

ESTD:1995

- One main difference is the number of timers.
- Here, each frame sent or resent needs a timer, which means that the timers need to be numbered (0, 1,2, and 3).
- The timer for frame0 starts at the first request, but stops when the ACK for this frame arrives.
- The timer for frame I starts at the second request, restarts when a NAK arrives, and finally stops when the last ACK arrives.
- The other two timers start when the corresponding frames are sent and stop at the last arrival event.

Selective Repeat Automatic Repeat Request



Piggybacking

- The three protocols discussed are all unidirectional:
- Data frames flow in only one direction although control information such as ACK and NAK frames can travel in the other direction.
- In real life, data frames are normally flowing in both directions: from node A to node B and from node B to node A.
- The control information also needs to flow in both directions. A technique called **piggybacking** is used to improve the efficiency of the bidirectional protocols.
- When a frame is carrying data from A to B, it can also carry control information about arrived (or lost) frames from B; when a frame is carrying data from B to A, it can also carry control information about the arrived (or lost) frames from A.

Random access

ESTD:1995

- In random access or contention methods, no station is superior to another station and none is assigned the control over another.
- No station permits, or does not permit, another station to send.
- At each instance, a station that has data to send uses a procedure defined by the protocol to make a decision on whether or not to send.
- This decision depends on the state of the medium (idle or busy).
- Two features give this method its name. **First,** there is no scheduled time for a station to transmit. Transmission is random among the stations. Hence this these methods are called *random access.*
- **Second,** no rules specify which station should send next. Stations compete with one another to access the medium. Hence the methods are also called *contention* methods

Random access

ESTD:1995

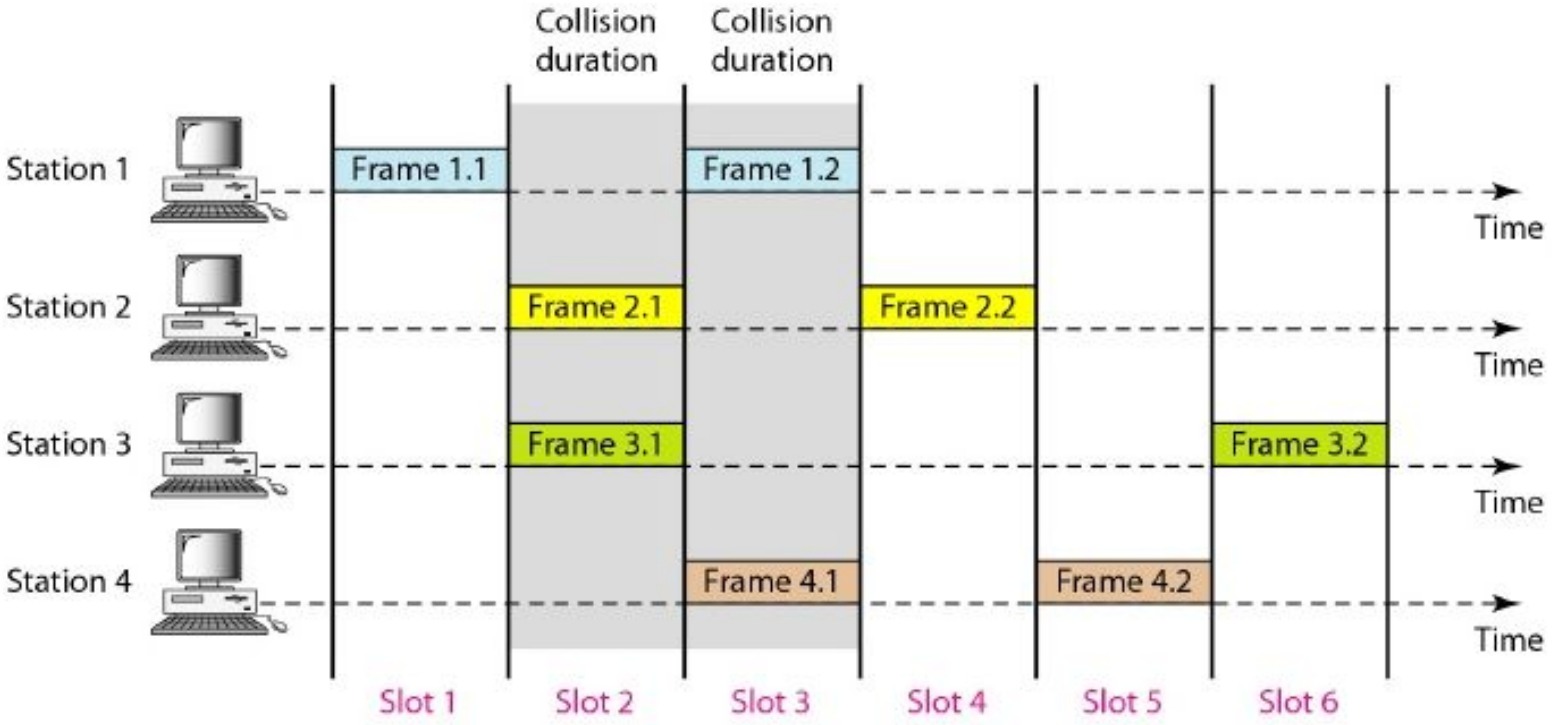
- To avoid access conflict or to resolve it when it happens, each station follows a procedure that answers the following questions:
    1. When can the station access the medium?
    2. What can the station do if the medium is busy?
    3. How can the station determine the success or failure of the transmission?
    4. What can the station do if there is an access conflict?
- The random-access methods have evolved the protocol known as ALOHA, which used a very simple procedure called multiple access (MA).
- This method was improved with the addition of a procedure that forces the station to sense the medium before transmitting called carrier sense multiple access.

Random access

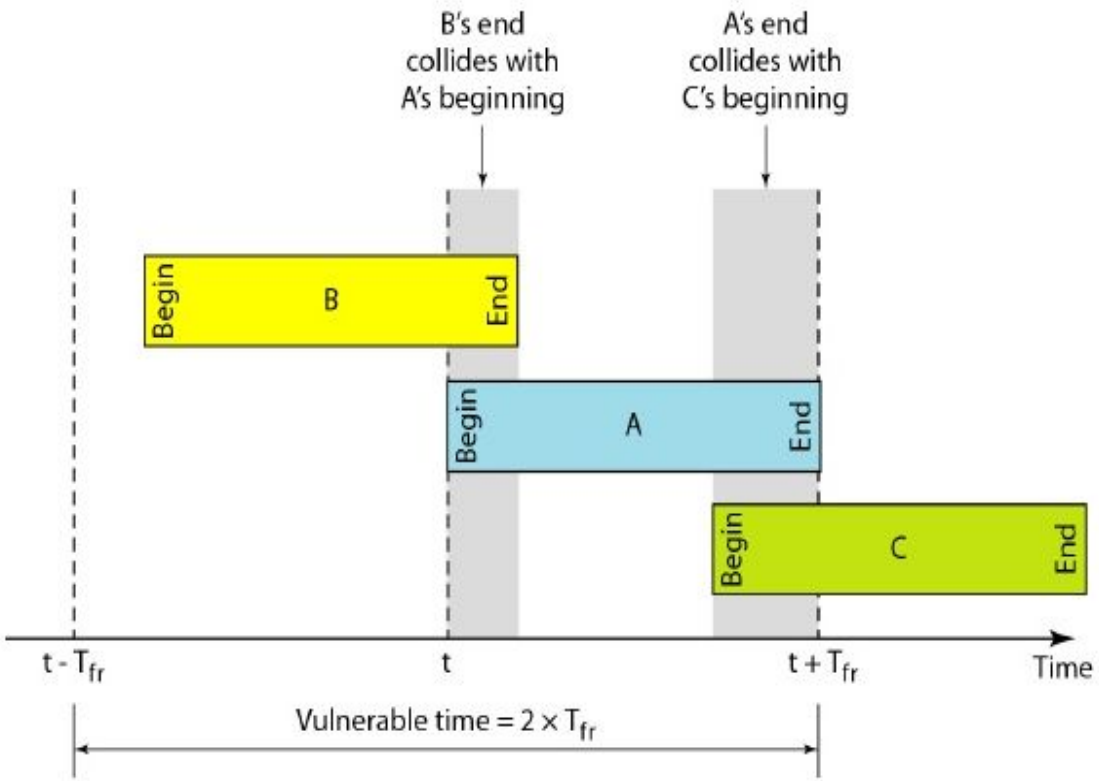
CSMA evolved into two parallel methods: carrier sense multiple access with collision detection (CSMA/CD) and carrier sense multiple access with collision avoidance *(CSMA/CA). CSMA/CD* tells the station what to do when a collision is detected. *CSMA/CA* tries to avoid the collision.

- **ALOHA**
  - ALOHA, the earliest random-access method, was developed at the University of Hawaii in early 1970.
  - It was designed for a radio (wireless) LAN, but it can be used on any shared medium. The medium is shared between the stations and hence collisions When a station sends data, another station may attempt to do so at the same time.
  - **Data garbled.**

Pure Aloha


Collision duration
Collision duration
Station 1
Station 2
Station 3
Station 4
Frame 1.1
Frame 1.2
Frame 2.1
Frame 2.2
Frame 3.1
Frame 3.2
Frame 4.1
Frame 4.2
Time
Time
Time
Time
Slot 1
Slot 2
Slot 3
Slot 4
Slot 5
Slot 6

Pure Aloha

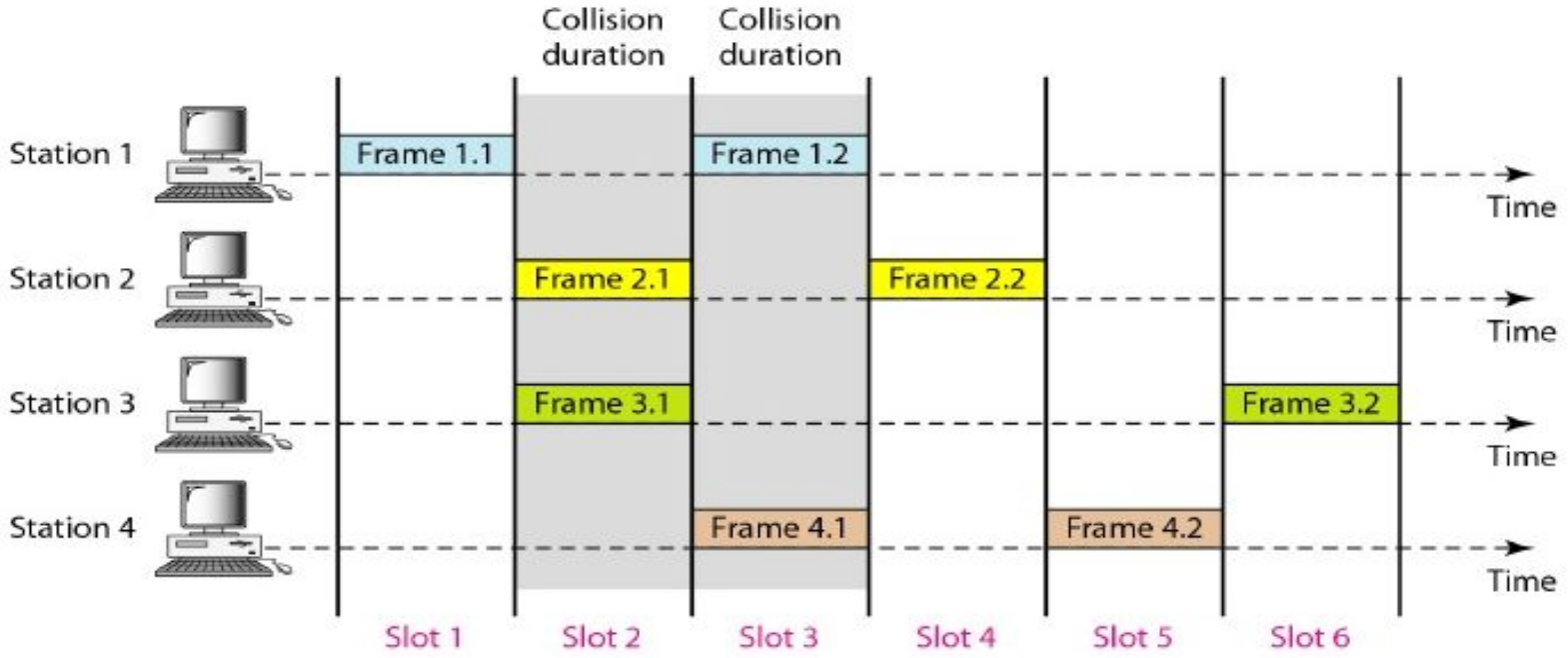


Vulnerable time for Pure Aloha

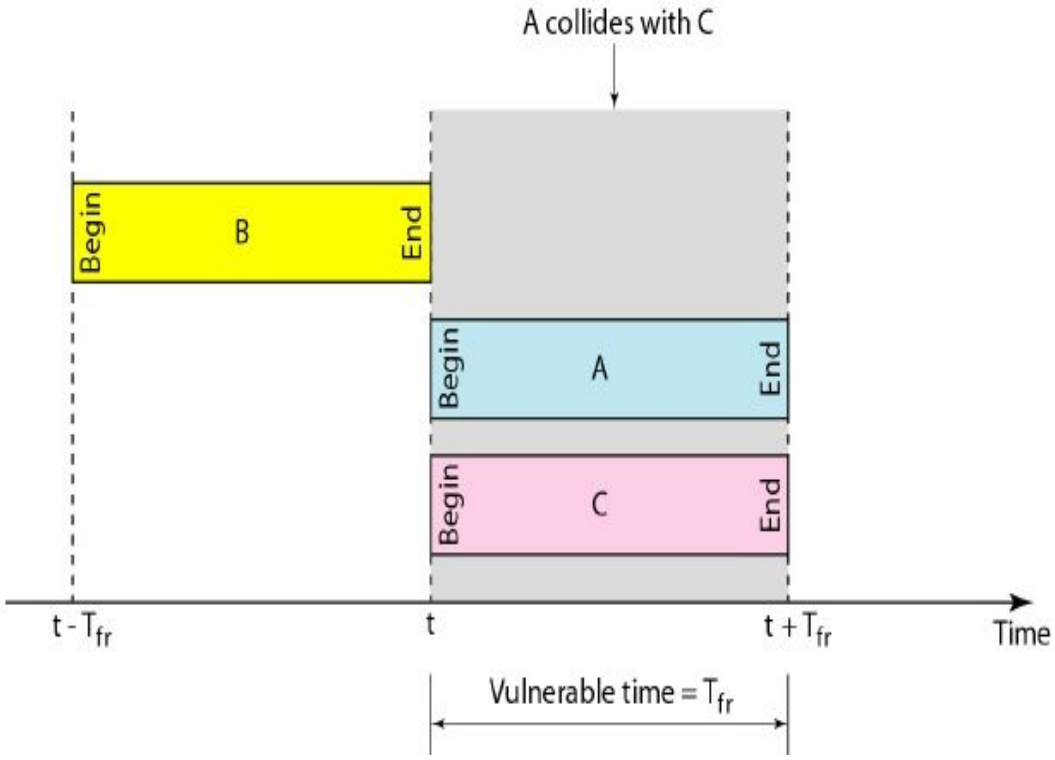
Average number of successful transmissions for pure ALOHA is $S = G * e^{-2G}$. The maximum throughput is 0.184, for G = 1.

Pure Aloha

- ***Slotted ALOHA***
- In slotted ALOHA we divide the time into slots of T_{fr} s and force the station to send only at the beginning of the time slot.



Pure Aloha



Average number of successful transmissions for slotted ALOHA is $S = G * e^{-G}$.
The maximum throughput is 0.364, for G = 1/2.

Vulnerable time for Slotted Aloha

WIT and WIL Scenario

Practice Questions

ESTD:1995

1. What are the three criteria necessary for an effective and efficient network?(2M)
2. What are the three fundamental characteristics determine the effectiveness of the data communication system?(2M)
3. Describe the goals of multiplexing?(8M)
4. Distinguish between synchronous and statistical TDM.(8M)
5. How do guided media differ from unguided media?
6. What is the purpose of cladding in an optical fiber?
7. How does sky propagation differ from line-of-sight propagation?
8. What is the difference between omnidirectional waves and unidirectional waves?

References

- http://highered.mheducation.com/sites/0072967757/student_view0/chapter1/multiple_choice_quiz.html
- http://citengg.blogspot.com/p/behrouz-forouzancomputer-networks4th.html
- https://study.com/academy/lesson/transmission-basics-in-networking-media-characteristics-connector-types.html
- https://www.geeksforgeeks.org/types-transmission-media/
- https://www.javatpoint.com/multiplexing-in-computer-network
- https://www.tutorialspoint.com/data_communication_computer_network/physical_layer_multiplexing.htm
- https://www.computernetworkingnotes.com/networking-tutorials/multiplexing-and-demultiplexing-explained-with-types.html

THANK YOU